BattleOps Week 5 Assignments

DAY 1: PDB STARVATION + FORCED TERMINATION CASCADE

Objective: Simulate what happens when graceful shutdown + PDB + CPU pressure collide → *cascading unavailability*.

SETUP: High Load Deployment

1. kubectl create ns battle-5
2. kubectl create deploy web --image=nginx -n battle-5
3. kubectl scale deploy web --replicas=5 -n battle-5

Add PDB: cat <<EOF | kubectl apply -n battle-5 -f -

- apiVersion: policy/v1
- kind: PodDisruptionBudget
- metadata:
- name: web-pdb
- spec:
- minAvailable: 4
- selector:
- matchLabels:
- app: web
- EOF

FAILURE 1: Force CPU Meltdown on Pods

- Inject CPU pressure in all pods:
- for p in $(kubectl get pod -n battle-5 -o name); do
- kubectl exec -n battle-5 $p -- sh -c "yes > /dev/null &"
- done

OBSERVE

kubectl top pods -n battle-5

Expect:

- 100% CPU usage
- Throttling
- Pods slow to respond

FAILURE 2: Drain Node to Trigger PDB Starvation

kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data

OBSERVE SYMPTOMS

Run: kubectl get ev -n battle-5 | grep -i evict || kubectl get pod -n battle-5 -w

Expected symptoms:

- Some pods evicted
- Then drain *hangs*
- PDB blocks eviction
- Kubelet retries → slow responsiveness

FIX: Scale replicas higher:kubectl scale deploy web --replicas=7 -n battle-5
Retry drain.

EXPLAIN WHAT HAPPENED

You must explain:

- Why PDB blocked completion
- Why CPU pressure slowed eviction
- Why readiness probes flapped
- Why cascading failures appeared

DAY 2: BACKUP SUCCESS BUT DATA LOST (Velero Corruption Simulation)

Objective: Reproduce the REAL WORLD INCIDENT: **“Backup succeeded but restore produced empty data.”**

SETUP: Deploy MySQL with real writes

kubectl apply -n battle-5 -f https://raw.githubusercontent.com/kubernetes/examples/master/mysql/mysql-statefulset.yaml

Generate constant writes:kubectl exec -n battle-5 -it mysql-0 -- sh -c "
while true; do mysql -uroot -ppassword -e 'INSERT INTO test.testtable VALUES (RAND())'; done"

TAKE VELERO BACKUP WITHOUT VOLUME SNAPSHOTS

velero backup create mysql-backup --include-namespaces battle-5
Velero says: **Completed**
But NO PVC data was captured.

FAILURE: Delete Namespace

kubectl delete ns battle-5

RESTORE

- velero restore create mysql-restore --from-backup mysql-backup🔍 **OBSERVE**
- kubectl get pods -n battle-5
- kubectl exec -n battle-5 mysql-0 -- mysql -uroot -ppassword -e "SELECT * FROM test.testtable LIMIT 10;"

Expected:

- Table empty
- Data missing
- Restore succeeded but data corrupted

FIX: Enable Restic or Snapshot Plugin

Reinstall Velero: velero install --use-restic …
Redo the test.

EXPLAIN

You must answer: **Why did the backup succeed but restore failed?**

(Correct answer: Kubernetes objects restored, but PVC block data was NOT included.)

DAY 3: CHAOS: NETWORK PARTITION + LATENCY CASCADE

Objective: Simulate real microservice degradation:

- partial network cuts
- packet delay
- cascading timeout failures

SETUP: 2 Services Communicating

1. Service A: kubectl run svc-a --image=nginx -n battle-5
2. Service B: kubectl run svc-b --image=nginx -n battle-5
3. Expose both: kubectl expose pod svc-a --port=80 -n battle-5
   kubectl expose pod svc-b --port=80 -n battle-5

FAILURE 1: Inject 80% Packet Loss between svc-a and svc-b

Pick svc-a pod: kubectl exec -n battle-5 svc-a -- tc qdisc add dev eth0 root netem loss 80%

OBSERVE

From inside svc-a: kubectl exec -n battle-5 svc-a -- curl -I svc-b

Expect:

- extremely slow
- intermittent timeouts
- HTTP incomplete responses

Check readiness: kubectl describe pod svc-a

Often → readiness fails → cascading issues.

FIX

Remove chaos: kubectl exec -n battle-5 svc-a -- tc qdisc del dev eth0 root

EXPLAIN

Given logs + events: **Explain how latency cascaded across service chain.**

DAY 4: MULTI-ZONE FAILURE SIMULATION (Zonal Outage)

Objective: Simulate a cloud zone outage and observe workload failure behavior.

SETUP: Multi-Zone Deployment

- kubectl apply -n battle-5 -f - <<EOF
- apiVersion: apps/v1
- kind: Deployment
- metadata:
- name: zone-test
- spec:
- replicas: 9
- selector:
- matchLabels:
- app: zone-test
- template:
- metadata:
- labels:
- app: zone-test
- spec:
- containers:
- - name: nginx
- image: nginx
- EOF

Identify nodes by zone: kubectl get nodes --show-labels | grep topology.kubernetes.io/zone

FAILURE: Blackout One Zone

Taint all nodes in zone A: kubectl taint node <node-in-zoneA> zone=down:NoExecute

OBSERVE

kubectl get pods -n battle-5 -o wide

Expected:

- All pods in zone A **evicted**
- Pods attempt to reschedule in remaining zones
- If insufficient capacity → pods stuck Pending
- If PVCs zonal → StatefulSets fail to recover

FIX

Add topology spread constraints:

- topologySpreadConstraints:
- - maxSkew: 1
- topologyKey: topology.kubernetes.io/zone
- labelSelector:
- matchLabels:
- app: zone-test

Redeploy: kubectl apply -f zone-test.yaml

EXPLAIN

You must explain:

- Why pods could not move
- What zonal PVC limitations caused
- Why auto-scaler did NOT react fast enough

DAY 5: RUNBOOK FAILURE DRILL + FAILOVER UNDER CHAOS

Objective: Simulate a real-world midnight outage with:

- incomplete runbook
- partial visibility
- degraded cluster
- incorrect assumptions

This is the hardest test.

SETUP: Blue/Green App

Deploy v1: kubectl create deploy app-v1 --image=nginx -n battle-5
kubectl expose deploy app-v1 --port=80 -n battle-5 --name=app

Deploy v2:kubectl create deploy app-v2 --image=nginx -n battle-5

FAILURE: Kill v1 but Hide Logs

kubectl scale deploy app-v1 --replicas=0 -n battle-5
kubectl delete pod <pod-of-v1> -n battle-5

Now simulate partial logging failure:kubectl delete deploy loki -n observability
Cluster loses logs.

TASK: Execute Failover With Missing Observability

Try to redirect service → v2: kubectl patch svc app -n battle-5 -p '{"spec":{"selector":{"app":"app-v2"}}}'

EXPECTED CHALLENGES

- DNS delay
- Old endpoints still cached
- readiness of v2 may flap
- old service mesh rules persist

FIX

1. Wait for endpoints to update: kubectl get endpoints app -n battle-5 -w
2. Force rollout refresh: kubectl rollout restart deploy app-v2 -n battle-5
3. Validate traffic path.

EXPLAIN

You must write a summary:

- What broke?
- What information was missing?
- Which runbook step failed?
- How you would rewrite the runbook?